

Website Performance Checklist

25 performance checks across images, CSS, JavaScript, fonts, caching, and Core Web Vitals — plus current Core Web Vitals targets, an image optimization guide, and a caching strategy overview.

Core Web Vitals Targets

Google measures these three metrics at the 75th percentile of real user visits (field data from the Chrome User Experience Report), not a single lab test — a page needs to perform well for the slower end of your real traffic, not just on a fast connection in your own testing.

Metric	What It Measures	Good	Needs Improvement	Poor
LCP (Largest Contentful Paint)	Loading speed — time until the largest visible element renders	$\leq 2.5s$	2.5s – 4.0s	$> 4.0s$
INP (Interaction to Next Paint)	Responsiveness — how quickly the page responds to user interactions, across the full session	$\leq 200ms$	200ms – 500ms	$> 500ms$
CLS (Cumulative Layout Shift)	Visual stability — how much visible content shifts unexpectedly	≤ 0.1	0.1 – 0.25	> 0.25

INP replaced First Input Delay (FID) as the official responsiveness metric in March 2024 — any guidance still referencing FID is out of date. Thresholds shown reflect Google's current published standard; verify at web.dev if this checklist is more than a year or two old.

The 25-Item Checklist

Images (5)

- [] 1. Modern formats (WebP or AVIF) used in place of JPEG/PNG where browser support allows
- [] 2. Lazy loading applied to below-the-fold images (native loading="lazy" or equivalent)
- [] 3. Responsive images served via srcset/sizes, not a single oversized file for all viewports
- [] 4. Images compressed appropriately for their actual display size, not just their source resolution
- [] 5. Explicit width and height attributes set on every image, to reserve layout space and prevent CLS

CSS (4)

- [] 6. CSS minified in production builds
- [] 7. Critical above-the-fold CSS inlined, with the remainder loaded asynchronously
- [] 8. Unused CSS removed or tree-shaken from the production bundle
- [] 9. Render-blocking stylesheets minimized, especially for content above the fold

JavaScript (4)

- [] 10. JavaScript minified and bundled appropriately for production
- [] 11. Non-critical scripts loaded with defer or async rather than blocking parse
- [] 12. Large bundles code-split so users only download what the current page actually needs
- [] 13. Unused JavaScript identified and removed — audit periodically, since it accumulates silently over time

Fonts (3)

- [] 14. font-display: swap (or equivalent) set to avoid invisible text while fonts load
- [] 15. Font files subsetting to only the character sets actually used, not the full default character range
- [] 16. Critical fonts preloaded with <link rel="preload"> so they're not discovered late in the loading sequence

Caching (4)

- [] 17. Appropriate Cache-Control headers set for static assets (long max-age for versioned files)
- [] 18. A CDN used to serve static assets from a location close to the visitor
- [] 19. Asset URLs versioned or fingerprinted, so long cache lifetimes don't serve stale content after a deploy
- [] 20. Service worker or equivalent caching layer considered for repeat-visit performance, where appropriate

Core Web Vitals — Specific Fixes (5)

- [] 21. The actual LCP element identified (via DevTools or field data) and its loading path specifically optimized
- [] 22. Layout space reserved for ads, embeds, and dynamically injected content to prevent late-loading CLS
- [] 23. Long JavaScript tasks broken up so the main thread stays free to respond to interactions (supports INP)
- [] 24. preconnect or dns-prefetch used for critical third-party origins (fonts, CDN, analytics)

[] 25. Render-blocking resources above the fold eliminated or deferred wherever they aren't strictly required for first paint

Image Optimization Guide

Images are usually the single largest contributor to page weight, and the checks above only tell you what to do, not how much it matters or in what order to tackle it.

Format	Best For	Notes
AVIF	Best compression available; photos and complex images	Excellent savings, but confirm browser support for your audience before relying on it exclusively
WebP	Broad modern support; photos and graphics	Good default choice — supported by all current major browsers
JPEG	Fallback for photos where WebP/AVIF isn't served	Still useful as a compatibility fallback via the <picture> element
SVG	Icons, logos, simple illustrations	Scales without quality loss and is typically tiny — don't use a raster format for simple vector graphics

Priority Order

If you can only do a few things: (1) convert to modern formats, (2) add explicit width/height to stop layout shift, (3) lazy-load below-the-fold images, (4) implement responsive srcset. In that order, each fix tends to matter less than the one before it for most sites.

Caching Strategy Overview

Caching strategy is really three separate decisions, often conflated into one — treat them separately.

Layer	What It Solves	Typical Approach
Browser caching	Repeat visits by the same user don't re-download unchanged assets	Long max-age on versioned/fingerprinted static assets
CDN / edge caching	First-visit latency for users far from your origin server	Serve static assets (and cacheable HTML where possible) from edge locations
Application-level caching	Reduces load on your backend for expensive or repeated computations	Server-side response caching, database query caching, or a service worker for offline/repeat-visit support

The most common caching mistake is caching HTML aggressively without a versioning or invalidation strategy — this causes users to see stale content after a deploy, which erodes trust in the site far more than a slightly slower load would have.